MixBytes()

DIA Oracle V3 Security Audit Report

MARCH 31, 2026

Table of Contents

1. Introduction **3**

1.1 Disclaimer 3

1.2 Executive Summary 3

1.3 Project Overview 5

1.4 Security Assessment Methodology 8

1.5 Risk Classification 10

1.6 Summary of Findings 11

2. Findings Report **13**

2.1 Critical 13

2.2 High 13

H-1 Single oracle with non-zero fairValue flips the sort dimension for all oracles 13

H-2 setDecimals callable by UPDATER_ROLE — precision change silently excludes oracle and corrupts historical data 14

2.3 Medium 15

M-1 Incorrect median calculation for numerator/denominator ratios 15

M-2 Early return in MedianPrice and VWAP methodologies bypasses threshold check 16

M-3 VolumeWeightedOracleMethodology does not calculate true VWAP 17

M-4 ValueStore.setValue accepts numerator=0, denominator=0 — zero denominator can reach consumers 18

M-5 getAggregatedVolume counts zero-volume oracles as valid 19

M-6 Equal timestamps accepted — duplicate entries consume ring buffer 20

M-7 Removing oracles/stores or setting threshold can make threshold unsatisfiable 21

M-8 ValueStoreRemoved event emits incorrect index after swap 22

M-9 Stale rawData persists after setValue overwrites the same key 23

2.4 Low 24

L-1 getValue returns fairValue for unrecognized action strings	24
L-2 Even-count median timestamp ignores the fresher middle element	25
L-3 V2-compatible update functions incompatible with VWAP methodology	26
L-4 Unsafe uint256 to uint128 truncation in volume aggregation and getValue	27
L-5 Inconsistent rounding in even-count median calculation	28
L-6 Code quality issues	29
L-7 Missing __AccessControl_init call in DIAOracleV3.initialize	30
L-8 Potential overflow in _insertionSort comparison	31
3. About MixBytes	**32**

1. Introduction

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

DIA Oracle V3 is a decentralized oracle system consisting of two independent modules: the decentral-feeder (on-chain aggregation of price data from multiple `DIAOracleV3` instances via pluggable methodologies) and the fair-value module (aggregation of fair value, USD value, and numerator/denominator ratios from multiple `ValueStore` contracts via median calculation). Both modules are designed as upgradeable contracts targeting EVM-compatible chains.

The audit identified 2 High, 9 Medium, and 8 Low severity findings. The most impactful issues relate to incorrect median calculation logic in the fair-value module, a sort dimension vulnerability that allows a single misconfigured oracle to corrupt aggregation results, and an overly permissive access control on `setDecimals` that can silently break the entire oracle pipeline.

The codebase was audited in 4 days by 4 auditors using a combination of manual review and automated tooling.

Several additional observations did not warrant standalone findings but are important for the project's long-term health:

`MAX_HISTORY_SIZE` **is** `immutable` **and baked into implementation bytecode.** If a new implementation is deployed with a different value (e.g. 200 instead of 100), existing ring buffer arrays in the proxy remain at their original length. The `writeIndex` will exceed the array bounds and `setValue` will revert for all existing keys. This must be accounted for in any upgrade strategy.

OpenZeppelin dependencies are not pinned. The project does not build from a clean clone — tests are written against OZ v5.5.0, but `forge install` pulls v5.6.1 which contains breaking changes, causing 3 out of 4 test suites to fail. **Update:** the developers have added a Foundry lock file in both repositories to pin dependency versions.

A malicious or malfunctioning oracle/ValueStore can DoS the aggregator. In the fair-value module, arithmetic overflow in the insertion sort comparison (`keyMain + 1` when `keyMain == type(uint256).max`) and in the freshness check (`ts + timeoutSeconds` overflow outside the `try/catch` scope) both cause a revert that cannot be caught. A single compromised ValueStore returning extreme values can permanently block `getMedianValues` for all keys. While this requires a compromised trusted source and is low probability, the lack of defensive handling is worth noting.

getValue **returns zeros for nonexistent keys** in both DIAOracleV3 and DIAOracleV3MetaFairValueField. Consumers that do not check the returned timestamp may silently accept a zero price as valid data.

Oracles with different decimals **are filtered out rather than normalized.** DIAOracleV3Meta excludes oracles whose decimals does not match the meta-contract's setting, rather than rescaling values to a common precision. This is a design choice, but it means adding an oracle with a different decimal configuration silently reduces the effective oracle set without any warning.

Decimals in DIAOracleV3MetaFairValueField **are handled by the off-chain service.** All assets and platforms are expected to use the same decimals, so on-chain normalization is not performed.

Sorting algorithm in MedianPriceMethodology **is mislabeled.** The comments refer to "bubble sort", but the implementation is actually selection sort (finds the minimum for each position rather than swapping adjacent elements). Either the comments should be corrected or the algorithm replaced with an actual bubble sort or a more gas-efficient alternative like insertion sort.

Timestamp semantics differ between modules by design. DIAOracleV3, the timestamp is provided by the caller and represents the moment the price was observed at the off-chain source (validated to be within 1 hour of block.timestamp). In ValueStore, the timestamp was previously set to block.timestamp automatically, but is now also provided by the caller and validated via _validateTimestamp (within 1 hour of block.timestamp, strictly monotonically increasing per key). Since the two modules are fully independent — different metacontracts, different data sources, different consumers — these timestamps never cross module boundaries or get compared to each other. The difference is a natural consequence of the different trust and data models.

ValueStore.setValue **now enforces strictly increasing timestamps per key.** The re-audit commit added a caller-provided timestamp parameter to setValue with a monotonicity check via _validateTimestamp. This means that when batch-loading or replaying historical data, entries must be submitted in chronological order — out-of-order submissions will revert. This ordering requirement should be documented for off-chain service operators to avoid unexpected failures during data ingestion.

Ring buffer data cannot be edited or deleted after submission. Once a feeder writes a value into the DIAOracleV3 ring buffer, there is no mechanism to retroactively correct or remove it. If an oracle submits incorrect data, the erroneous entry remains in the history until it is naturally overwritten by newer entries or expires via the timeout. Instead of deleting values, the expected mitigation is to remove or disable the misbehaving feeder/node operator so it stops submitting incorrect data.

The off-chain feeder infrastructure, deployment scripts, and upgrade migration procedures were out of scope of this audit.

Overall, the codebase is well-structured and modular, but would benefit from pinning dependency versions, adding input validation on admin setters (bounds checks on threshold, windowSize, timeoutSeconds), improving test coverage for edge cases (even-length arrays, zero-volume entries, mixed-decimal oracles), and fixing the issues presented in the report below.

1.3 Project Overview

Summary

Title	Description
Client	DIA
Category	Oracle
Project	DIA Oracle V3
Type	Solidity
Platform	EVM
Timeline	04.03.2026 - 20.03.2026

Scope of Audit

File	Link
decentral-feeder/contracts/DIAOracleV3.sol	contracts/DIAOracleV3.sol
decentral-feeder/contracts/DIAOracleV3Meta.sol	contracts/DIAOracleV3Meta.sol
decentral-feeder/contracts/methodologies/MedianPriceMethodology.sol	contracts/methodologies/MedianPriceMethodology.sol
decentral-feeder/contracts/methodologies/VolumeWeightedAveragePriceMethodology.sol	contracts/methodologies/VolumeWeightedAveragePriceMethodology.sol
fair-value/contracts/metacontract/DIAOracleV3MetaFairValueField.sol	contracts/metacontract/DIAOracleV3MetaFairValueField.sol
fair-value/contracts/valuestore/ValueStore.sol	contracts/valuestore/ValueStore.sol

Versions Log

Date	Commit Hash	Note
04.03.2026	15e9049f57602bcb673fbcc8006ad6076470f8f4	Initial Commit (decentral-feeder)
04.03.2026	00bf59d5bd7153a549cedb09712eacd1caf74a2a	Initial Commit (fair-value)
18.03.2026	ff2ea102744ba3e027a23234397bbd92aae747d2	Commit for Re-audit (decentral-feeder)
18.03.2026	9cab374be377e55cad72c3d6c55a1a307735bea9	Commit for Re-audit (fair-value)
20.03.2026	38753f54d6e770e92d2b1d440b1ff9a2f5b96240	Commit with Updates (decentral-feeder)
20.03.2026	0789e7c67a7684b7747bdbf452326a2e0a79ba6f	Commit with Updates (fair-value)

Mainnet Deployments

The deployed contracts – DIAOracleV3, DIAOracleV3Meta, MedianPriceMethodology, DIAOracleV3MetaFairValueField, and ValueStore – were verified against the audited sources: bytecode and configuration parameters match.
Both ERC1967Proxy deployments – 0xf9AD4E64206F943708dFa6d5eFf0729e4095eD56 (proxying DIAOracleV3) and 0x3C3715bc53dD3Ba2E60DE7611F7F6C449641e5F9 (proxying ValueStore) – use identical unmodified OpenZeppelin v5.5.0 proxy sources and are not vulnerable to a Clandestine Proxy-in-the-Middle of Proxy attack – the constructor atomically sets the implementation and executes initialize() via delegatecall, confirmed by Initialized and Upgraded events emitted in each deployment transaction, leaving no window for front-running.
Currently, timeoutSeconds and windowSize in DIAOracleV3Meta are both set to 0, and timeoutSeconds in DIAOracleV3MetaFairValueField is also set to 0, meaning the meta-oracle will reject all oracle values as expired and return no valid aggregated price until these parameters are configured.
The OpenZeppelin dependencies are not pinned to a stable release tag. We recommend pinning dependencies to a stable version to ensure reproducible builds and consistency between

audited and deployed code.

File	Address	Blockchain
MedianPriceMethodology	0x551BC8...15232b97	DIA LaserNet
DIAOracleV3Meta	0xBd82Fc...cFb62D7d	DIA LaserNet
ERC1967Proxy	0xf9AD4E...4095eD56	DIA LaserNet
DIAOracleV3	0x042e72...7551Eaf8	DIA LaserNet
DIAOracleV3MetaFairValueField	0x5aCEc1...2410c0CC	DIA LaserNet
ERC1967Proxy	0x3C3715...9641e5F9	DIA LaserNet
ValueStore	0x0237Af...eB0054D8	DIA LaserNet

1.4 Security Assessment Methodology

Project Flow

Stage	Scope of Work
Interim audit	**Project Architecture Review:** • Review project documentation • Conduct a general code review • Perform reverse engineering to analyze the project's architecture based solely on the source code • Develop an independent perspective on the project's architecture • Identify any logical flaws in the design OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROJECT AND IDENTIFY POTENTIAL SECURITY RISKS.
	Code Review with a Hacker Mindset: • Each team member independently conducts a manual code review, focusing on identifying unique vulnerabilities. • Perform collaborative audits (pair auditing) of the most complex code sections, supervised by the Team Lead. • Develop Proof-of-Concepts (PoCs) and conduct fuzzing tests using tools like Foundry, Hardhat, and BOA to uncover intricate logical flaws. • Review test cases and in-code comments to identify potential weaknesses. OBJECTIVE: IDENTIFY AND ELIMINATE THE MAJORITY OF VULNERABILITIES, INCLUDING THOSE UNIQUE TO THE INDUSTRY.
	Code Review with a Nerd Mindset: • Conduct a manual code review using an internally maintained checklist, regularly updated with insights from past hacks, research, and client audits. • Utilize static analysis tools (e.g., Slither, Mythril) and vulnerability databases (e.g., Solodit) to uncover potential undetected attack vectors. OBJECTIVE: ENSURE COMPREHENSIVE COVERAGE OF ALL KNOWN ATTACK VECTORS DURING THE REVIEW PROCESS.

Stage	Scope of Work
	Consolidation of Auditors' Reports: · Cross-check findings among auditors · Discuss identified issues · Issue an interim audit report for client review OBJECTIVE: COMBINE INTERIM REPORTS FROM ALL AUDITORS INTO A SINGLE COMPREHENSIVE DOCUMENT.
Re-audit	**Bug Fixing & Re-Audit:** · The client addresses the identified issues and provides feedback · Auditors verify the fixes and update their statuses with supporting evidence · A re-audit report is generated and shared with the client OBJECTIVE: VALIDATE THE FIXES AND REASSESS THE CODE TO ENSURE ALL VULNERABILITIES ARE RESOLVED AND NO NEW VULNERABILITIES ARE ADDED.
Final audit	**Final Code Verification & Public Audit Report:** · Verify the final code version against recommendations and their statuses · Check deployed contracts for correct initialization parameters · Confirm that the deployed code matches the audited version · Issue a public audit report, published on our official GitHub repository · Announce the successful audit on our official X account OBJECTIVE: PERFORM A FINAL REVIEW AND ISSUE A PUBLIC REPORT DOCUMENTING THE AUDIT.

1.5 Risk Classification

Severity Level Matrix

Severity	Impact: High	Impact: Medium	Impact: Low
Likehood: High	Critical	High	Medium
Likehood: Medium	High	Medium	Low
Likehood: Low	Medium	Low	Low

Impact

- **High** – Theft from 0.5% OR partial/full blocking of funds (>0.5%) on the contract without the possibility of withdrawal OR loss of user funds (>1%) who interacted with the protocol.
- **Medium** – Contract lock that can only be fixed through a contract upgrade OR one-time theft of rewards or an amount up to 0.5% of the protocol's TVL OR funds lock with the possibility of withdrawal by an admin.
- **Low** – One-time contract lock that can be fixed by the administrator without a contract upgrade.

Likelihood

- **High** – The event has a 50-60% probability of occurring within a year and can be triggered by any actor (e.g., due to a likely market condition that the actor cannot influence).
- **Medium** – An unlikely event (10-20% probability of occurring) that can be triggered by a trusted actor.
- **Low** – A highly unlikely event that can only be triggered by the owner.

Action Required

- **Critical** – Must be fixed as soon as possible.
- **High** – Strongly advised to be fixed to minimize potential risks.
- **Medium** – Recommended to be fixed to enhance security and stability.
- **Low** – Recommended to be fixed to improve overall robustness and effectiveness.

Finding Status

- **Fixed** – The recommended fixes have been implemented in the project code and no longer impact its security.
- **Partially Fixed** – The recommended fixes have been partially implemented, reducing the impact of the finding, but it has not been fully resolved.
- **Acknowledged** – The recommended fixes have not yet been implemented, and the finding remains unresolved or does not require code changes.

1.6 Summary of Findings

Findings Count

Severity	Count
Critical	0
High	2
Medium	9
Low	8

Findings Statuses

ID	Finding	Severity	Status
H-1	Single oracle with non-zero fairValue flips the sort dimension for all oracles	High	Fixed
H-2	setDecimals callable by UPDATER_ROLE — precision change silently excludes oracle and corrupts historical data	High	Fixed
M-1	Incorrect median calculation for numerator/denominator ratios	Medium	Acknowledged
M-2	Early return in MedianPrice and VWAP methodologies bypasses threshold check	Medium	Fixed
M-3	VolumeWeightedOracleMethodology does not calculate true VWAP	Medium	Acknowledged
M-4	ValueStore.setValue accepts numerator=0, denominator=0 — zero denominator can reach consumers	Medium	Acknowledged
M-5	getAggregatedVolume counts zero-volume oracles as valid	Medium	Fixed
M-6	Equal timestamps accepted — duplicate entries consume ring buffer	Medium	Fixed
M-7	Removing oracles/stores or setting threshold can make threshold unsatisfiable	Medium	Fixed

M-8	ValueStoreRemoved event emits incorrect index after swap	Medium	Fixed
M-9	Stale rawData persists after setValue overwrites the same key	Medium	Fixed
L-1	getValue returns fairValue for unrecognized action strings	Low	Fixed
L-2	Even-count median timestamp ignores the fresher middle element	Low	Fixed
L-3	V2-compatible update functions incompatible with VWAP methodology	Low	Acknowledged
L-4	Unsafe uint256 to uint128 truncation in volume aggregation and getValue	Low	Fixed
L-5	Inconsistent rounding in even-count median calculation	Low	Fixed
L-6	Code quality issues	Low	Fixed
L-7	Missing __AccessControl_init call in DIAOracleV3.initialize	Low	Fixed
L-8	Potential overflow in _insertionSort comparison	Low	Fixed

2. Findings Report

2.1 Critical

Not Found

2.2 High

H-1	Single oracle with non-zero fairValue flips the sort dimension for all oracles		
Severity	High	**Status**	Fixed in 9cab374b

Description

In DIAOracleV3MetaFairValueField._sortValues, the sort key is chosen globally based on whether fairSum is non-zero. If even one ValueStore returns a non-zero fairValue while others return zero (e.g. because the asset has no fair value in those stores), the entire array is sorted by fairValues — a field where most elements are zero. Zero-valued entries cluster at the bottom of the sorted array, and the median position lands on an arbitrary oracle whose result bears no relation to the actual median of the data. The usdValue, numerator, and denominator fields — which are reordered in parallel — end up in an equally meaningless order. A single misconfigured or partially-reporting ValueStore controls the sort dimension for the whole aggregation, producing an incorrect oracle output.

Recommendation

Sort by the ratio or by individual oracle fair values rather than using a global sum-based heuristic. Alternatively, filter out oracles that lack a fairValue before sorting, so that zero entries do not pollute the median calculation.

Client's Commentary:

Fixed by removing leading zeros in median calculation.

5ea74a49

H-2	setDecimals callable by UPDATER_ROLE — precision change silently excludes oracle and corrupts historical data		
Severity	High	**Status**	Fixed in ff2ea102

Description

In DIAOracleV3, the setDecimals function is gated by UPDATER_ROLE. Since DIAOracleV3Meta.getValue filters oracles by matching decimals, a decimals change silently excludes the oracle from aggregation. If this drops the count of matching oracles below threshold, all getValue calls start reverting. On top of that, existing historical values in the ring buffer remain stored with the old precision but are now interpreted under the new one, silently corrupting price data for all consumers.

Recommendation

We recommend disallowing changing decimals once any values have been written to the oracle.

Client's Commentary:

fixed as per recommendation

7e1fc667

2.3 Medium

M-1	Incorrect median calculation for numerator/denominator ratios		
Severity	Medium	**Status**	Acknowledged

Description

In DIAOracleV3MetaFairValueField._calculateMedian, when there is an even number of ValueStores, the median of numerator and denominator is calculated by averaging them independently:

```
uint256 sumNum = nums[mid1] + nums[mid2];
numerator = (sumNum + 1) / 2;

uint256 sumDen = dens[mid1] + dens[mid2];
denominator = (sumDen + 1) / 2;
```

This produces (a+c)/(b+d), which is not equal to the median of two ratios (a/b + c/d) / 2 = (ad + bc) / (2bd). When the two middle oracles have different scales for their numerator and denominator, the resulting ratio can be meaningfully incorrect.
At the same time, it should be noted that if numerator and denominator are calculated either as an arithmetic mean or using the formula above, they lose their physical meaning. They no longer represent the amount of assets anywhere or some value in USD.

Recommendation

Compute the median of ratios correctly, e.g. as (ad + bc) / (2bd), or document the current behavior as an intentional simplification and its limitations.
Additionally, consider the following approach as an option: instead of calculating numerator and denominator, use the values from the store with the highest timestamp. This would preserve a certain physical meaning.

Client's Commentary:

Client: *The suggestion does not work because d (and b) can be zero for some use-cases.*
Potentially losing physical meaning is the tradeoff for covering a wide range of use-cases here.
MixBytes: *In the case where both numerator and denominator are zero (0/0), the median could be taken as 0/0 as well. In all other cases, the mathematically correct formula (ad + bc) / (2bd) should be used, since b and d are non-zero there. This would preserve correctness without breaking the zero edge case.*
Client: *As discussed:*
This is actually not a bug but a feature. I should have mentioned that this contract will be used for a wide range of use cases, not only plain fair-value where fairValue=numerator/denominator. Instead, it will also cover cases where fairValue is not well-defined (and set to 0), and only the denominator will be non-zero. (For instance for assets backed by several different reserve currencies.)
In case numerator and denominator are non-zero and fairValue is as well, you're right that median(numerator)/median(denominator)!=median(fairValue). However, in this case the client can just use median(fairValue) and ignore the quotient.
This is why we need medians of numerator and denominator as independent quantities.

M-2	Early return in MedianPrice and VWAP methodologies bypasses threshold check		
Severity	Medium	**Status**	Fixed in ff2ea102

Description

In MedianPriceMethodology._aggregateMedianMedian and VolumeWeightedAveragePriceMethodology.calculateValue, if the incoming oracles array is empty (for instance, because no oracle matched by decimals in DIAOracleV3Meta), both methods return (0, block.timestamp) before the threshold check is reached. Returning a zero price with a valid current timestamp is incorrect — downstream consumers have no way to distinguish this from a legitimate price update, and the threshold mechanism that is supposed to guard against insufficient data is entirely bypassed.

Recommendation

Move the threshold check before the early return, or revert explicitly when the oracles array is empty.

Client's Commentary:

fixed as per recommendation

6627c4b7

M-3	VolumeWeightedOracleMethodology does not calculate true VWAP		
Severity	Medium	**Status**	Acknowledged

Description

Note: VolumeWeightedOracleMethodology was not in the original scope of this audit but was reviewed as part of the methodology contracts analysis.

In VolumeWeightedOracleMethodology._calculateOracleData, the per-oracle step computes a simple average price (priceSum / validCount) and then the cross-oracle step weights each oracle's average by its total volume. True VWAP weights each individual price entry by its own volume: sum(price_i * volume_i) / sum(volume_i). When volumes within a single oracle are unevenly distributed across entries (e.g. one entry with high volume at $100 and another with low volume at $200), the simple average ($150) weighted by total volume gives a different — and less accurate — result than weighting each price individually.

Recommendation

Weight each price entry by its individual volume rather than averaging prices first and weighting by total volume afterwards.

Client's Commentary:

Smart Contract is not required and complete

M-4	ValueStore.setValue accepts numerator=0, denominator=0 — zero denominator can reach consumers		
Severity	Medium	**Status**	Acknowledged

Description

In ValueStore.setValue, the validation only reverts when numerator != 0 && denominator == 0. The case where both are zero passes through. These entries represent "no data" (the off-chain scraper zeroes both fields on nil), but they participate in _collectValues and the median calculation as valid data points. With an even number of oracles, averaging a (0,0) entry with a real one can yield denominator=0 on output, causing division by zero in downstream consumers that use the ratio.

Recommendation

Either filter out entries with denominator == 0 in _collectValues, or reject (0,0) pairs in ValueStore.setValue.

Client's Commentary:

Client: *This is on purpose and serves some edge cases needed for initialization of products based on the fair-value feed.*

MixBytes: *If the intent is to allow `denominator=0` for initialization edge cases while continuing to use the `(a+c)/(b+d)` averaging formula, then in the case where both numerator and denominator are zero (0/0), the median could be taken as 0/0 as well. In all other cases, the mathematically correct formula `(ad + bc) / (2bd)` should be used, since b and d are guaranteed to be non-zero there. This would prevent a zero denominator from propagating to consumers through the averaging logic while still supporting the initialization use case.*

M-5	getAggregatedVolume counts zero-volume oracles as valid		
Severity	Medium	**Status**	Fixed in ff2ea102

Description

In DIAOracleV3Meta.getAggregatedVolume, oracles whose latest value has volume=0 (e.g. those updated via V2 functions) are still counted in validOracleCount. The returned count is meant to indicate how many oracles contributed to the aggregated volume, but it includes oracles that contributed nothing. Consumers relying on this count to assess data quality or quorum are misled into believing more sources provided volume data than actually did.

Recommendation

Skip oracles with zero volume when incrementing validOracleCount, or document that the count reflects oracles with fresh data regardless of whether they contributed volume.

Client's Commentary:

fixed as per recommendation

4e353330

M-6	Equal timestamps accepted — duplicate entries consume ring buffer		
Severity	Medium	**Status**	Fixed in ff2ea102

Description

In DIAOracleV3._validateTimestamp, the monotonicity check uses strict less-than: if (timestamp < existingTimestamp). This means timestamp == existingTimestamp is accepted. Each such call adds a new entry to the ring buffer. If an off-chain updater has retry logic and resubmits data with the same timestamp, the buffer fills with duplicates, evicting real price history. The NatSpec states "monotonically increasing" but the implementation enforces "non-decreasing".

Recommendation

Use <= instead of < in the timestamp comparison to enforce strictly increasing timestamps as documented.

Client's Commentary:

fixed as per recommendation

ffd8915d

M-7	Removing oracles/stores or setting threshold can make threshold unsatisfiable		
Severity	Medium	Status	Fixed in 0789e7c6

Description

In DIAOracleV3Meta.removeOracle, DIAOracleV3MetaFairValueField.removeValueStore, and both contracts' setThreshold, there is no validation that threshold <= numOracles/numValueStores after the operation. If the owner removes an oracle when threshold == numOracles, or sets threshold above the current count, every subsequent getValue/getMedianValues call reverts with ThresholdNotMet. The system becomes fully inoperable until the owner either adds a new source or lowers the threshold.

Recommendation

Add a guard in removeOracle/removeValueStore that reverts if the remaining count would fall below threshold. Similarly, cap setThreshold at the current oracle/store count.

Client's Commentary:

Client: *fixed as per recommendation*

83aa16d4

MixBytes: *The fix was only applied in the `decentral-feeder` (`DIAOracleV3Meta`). The same issue remains in the `fair-value` – `DIAOracleV3MetaFairValueField.removeValueStore` and `setThreshold` still lack the guard, so removing a ValueStore or raising the threshold can still make it unsatisfiable.*

Client: *Added same check in fair value field*

b0c984ae

M-8	ValueStoreRemoved event emits incorrect index after swap		
Severity	Medium	**Status**	Fixed in 9cab374b

Description

In DIAOracleV3MetaFairValueField.removeValueStore, when a store is removed, the last element is swapped into the removed position and the array is shortened. The ValueStoreRemoved event is emitted with the index parameter, but at that point the index now refers to the swapped-in store, not the one that was removed. Off-chain indexers and monitoring tools that rely on this event to track which store was removed will associate the wrong store with the removal.

Recommendation

Emit the event before performing the swap, or include both the removed store address and the index in the event for unambiguous identification.

Client's Commentary:

fixed as per recommendation

66c576e1

M-9	Stale rawData persists after setValue overwrites the same key		
Severity	Medium	**Status**	Fixed in ff2ea102

Description

In DIAOracleV3, calling setRawValue stores arbitrary bytes in the rawData mapping for a given key. If setValue is subsequently called for the same key, it updates the price history but does not clear the associated rawData. The old raw data remains accessible via getRawData, creating a mismatch between the current price entry and the stale raw payload. Consumers or off-chain services that correlate getRawData with the latest value will operate on outdated information.

Recommendation

Clear rawData[key] inside setValue and setMultipleValues, or document that raw data is independent of the value history and may become stale.

Client's Commentary:

Raw Data is cleared as per recommendation for v2 update function

a50f1bbd

2.4 Low

L-1	getValue returns fairValue for unrecognized action strings		
Severity	Low	Status	Fixed in 9cab374b

Description

In DIAOracleV3MetaFairValueField.getValue, if the action portion of the key does not match any of the four known hashes (fairValue, usdValue, numerator, denominator), the function silently falls through and returns fairValue. A consumer querying a mistyped key like "farValue:BTC/USD" receives a valid-looking response with no indication that the requested field is not what they intended.

Recommendation

Revert with a descriptive error when the action hash does not match any known field.

Client's Commentary:

Revert with invalid action when action hash does not match

f339e59a

L-2	Even-count median timestamp ignores the fresher middle element		
Severity	Low	Status	Fixed in 9cab374b

Description

In DIAOracleV3MetaFairValueField._calculateMedian, the comment says "Use max timestamp between selected one" but the code unconditionally takes timestamps[mid2] without comparing it to timestamps[mid1]. After sorting by fairValue, timestamps are reordered as a parallel array — mid2 corresponds to the higher fairValue, not necessarily the higher timestamp. When mid1 is fresher than mid2, the returned timestamp is staler than it should be, which can falsely trigger staleness checks in downstream protocols.

Recommendation

Take max(timestamps[mid1], timestamps[mid2]) as the comment already describes.

Client's Commentary:

use max timestamp, as per recommendation

a56acca1

L-3	V2-compatible update functions incompatible with VWAP methodology		
Severity	Low	**Status**	Acknowledged

Description

In DIAOracleV3, setValue and setMultipleValues store entries with volume=0 for backward compatibility. VolumeWeightedAveragePriceMethodology._calculateOracleVWAP skips entries with zero volume. If an oracle is updated exclusively through V2 functions, all its entries are skipped, the oracle is marked invalid, and it drops out of the aggregation entirely. When enough oracles use V2 functions, the valid count falls below threshold and getValue reverts. The system provides no error or warning about this incompatibility — the failure surfaces only as a generic ThresholdNotMet.

Recommendation

Document the requirement for volume data when using the VWAP methodology. Consider adding a distinct revert (e.g. NoVolumeData) when all oracles return valid=false due to missing volume, to distinguish this case from genuinely insufficient data.

Client's Commentary:

This is expected. VWAP will be used once all V2 contracts are upgraded to V3.

L-4	Unsafe uint256 to uint128 truncation in volume aggregation and getValue		
Severity	Low	**Status**	Fixed in 0789e7c6

Description

In DIAOracleV3Meta.getValueWithVolume, the accumulated volumeSum (uint256) is cast to uint128 via totalVolume = uint128(volumeSum) without an overflow check. Solidity 0.8.x does not revert on explicit type conversions — if the sum exceeds type(uint128).max, the value is silently truncated to the lower 128 bits. The same pattern appears in getAggregatedVolume. In DIAOracleV3MetaFairValueField.getValue, median results computed as uint256 are returned as uint128 with the same unchecked cast.

Recommendation

Use SafeCast.toUint128() or add an explicit require(volumeSum <= type(uint128).max) before the cast.

Client's Commentary:

Client: *Fixed as per recommendation 2991338a*

MixBytes: *The fix was only applied in the `decentral-feeder` (`DIAOracleV3Meta`). The unsafe `uint256` to `uint128` cast in `DIAOracleV3MetaFairValueField.getValue` in the `fair-value` module remains unfixed.*

Client: *SafeCast added in fair value field*

26b35c98

L-5	Inconsistent rounding in even-count median calculation		
Severity	Low	**Status**	Fixed in ff2ea102

Description

In MedianPriceMethodology and VolumeWeightedAveragePriceMethodology, when the number of values is even, the median is computed as (a + b) / 2, which rounds down. In contrast, DIAOracleV3MetaFairValueField._calculateMedian uses (sum + 1) / 2, which rounds up. The practical impact is minimal (1 wei difference), but the inconsistency across modules means the same input data can produce different prices depending on which aggregation path is used.

Recommendation

Use the same rounding approach ((sum + 1) / 2) across all median calculations for consistency.

Client's Commentary:

Use round up in every methodologies

b8095be9

L-6	Code quality issues		
Severity	Low	Status	Fixed in ff2ea102

Description

Several code quality issues were identified across the codebase:

Incorrect storage layout comment in DIAOracleV3**:** MAX_HISTORY_SIZE is listed as occupying slot 0, but it is declared immutable and lives in bytecode, not storage. The decimals variable (which does occupy storage) is missing from the comment. A developer planning a storage-compatible upgrade could misidentify slot boundaries.

getRawDataFromOracle **uses wrong error name:** reverts with InvalidHistoryIndex when the actual issue is an invalid oracle index. Confusing for debugging.

Unused ThresholdNotMet **error in** DIAOracleV3Meta**:** declared but never used — threshold checks happen inside methodology contracts.

Unused MAX_ALLOWED_HISTORY_SIZE **constant in** DIAOracleV3**:** declared but never referenced. Dead code.

Ring buffer first-write gas cost: _addToHistory allocates 100 ValueEntry structs via push in a loop on first write (~300 SSTORE to new slots, ~6M gas). Consider lazy initialization or a separate admin function to pre-allocate.

Double iteration over oracles in DIAOracleV3Meta.getValue**:** two passes with external getDecimals() calls (once to count, once to populate) instead of a single pass with a dynamic array.

uint128 **loop counter in** setMultipleValues**:** keys.length is uint256 but the loop variable i is uint128. Silently wraps for arrays longer than type(uint128).max. Unreachable in practice due to gas limits, but inconsistent with all other loops in the contract.

Recommendation

Address each item individually: fix the storage layout comment, rename the error, remove dead code, and consider gas optimizations for buffer initialization and oracle iteration.

Client's Commentary:

Fixed storage slot comment

2408d278

Added new error InvalidOracleIndex

37b1fe87

Removed unused error

06389114

Removed MAX_ALLOWED_HISTORY_SIZE

f3a35c65

use uint256 as iterator

ed33d5b5

removed double iteration

854f71d2

L-7	Missing __AccessControl_init call in DIAOracleV3.initialize		
Severity	Low	**Status**	Fixed in 38753f54

Description

DIAOracleV3.initialize does not call __AccessControl_init(). While this function is currently a no-op in OpenZeppelin v5.5, omitting it breaks the standard upgradeable initialization convention and creates a risk if a future version adds logic to it. Note: __UUPSUpgradeable_init() no longer exists in OpenZeppelin v5.5 (the contract is no longer transpiled), so its absence is correct.

Additionally, both repositories import UUPSUpgradeable and Initializable from @openzeppelin/contracts-upgradeable, which in v5.5 are merely aliases redirecting to @openzeppelin/contracts. These contracts were moved to the non-upgradeable package because UUPSUpgradeable is stateless (only an immutable) and Initializable uses ERC-7201 namespaced storage that does not require the transpilation process. Per the OpenZeppelin v5.5.0 changelog, these aliases will be removed in the next major release. The imports should be updated to reference @openzeppelin/contracts directly to avoid breakage on upgrade.

Recommendation

Add __AccessControl_init() to DIAOracleV3.initialize. Update imports of UUPSUpgradeable and Initializable to use @openzeppelin/contracts instead of @openzeppelin/contracts-upgradeable, as the aliases are deprecated and will be removed.

Client's Commentary:

Address accesscontrol init and fixed import 38753f54

https://github.com/diadata-org/fair-value/commits/oraclev3auditfix/

L-8	Potential overflow in `_insertionSort` comparison		
Severity	Low	**Status**	Fixed in 0789e7c6

Description

In `DIAOracleV3MetaFairValueField._insertionSort`, the comparison `main[prev] < keyMain + 1` can overflow if `keyMain` equals `type(uint256).max`. Although this value is unlikely under normal operation, a single compromised or malfunctioning ValueStore returning `type(uint256).max` as a fair value would cause an arithmetic overflow in the sort, reverting `getMedianValues` for all keys and effectively DoS-ing the entire fair-value aggregation.

Recommendation

Replace `main[prev] < keyMain + 1` with the equivalent `main[prev] <= keyMain`, which avoids the addition entirely and eliminates the overflow risk.

Client's Commentary:

Added this change as per recommendation

0558b2e0

3. About MixBytes

MixBytes is a leading provider of smart contract auditing and blockchain security research, helping Web3 projects enhance their security and reliability.

Since its inception, MixBytes has been dedicated to safeguarding innovation in **DeFi** through rigorous audits and cutting-edge technical research.

Our team brings together experienced engineers, security auditors, and blockchain researchers with deep expertise in smart contract security and protocol design.

With years of proven experience in Web3, MixBytes combines in-depth technical excellence with a proactive, security-first approach.

Why MixBytes

- **Proven Track Record:** Trusted by leading blockchain protocols such as **Lido**, **Aave**, **Curve**, **1inch**, **Fluid**, **Gearbox**,**Resolv**, and others. MixBytes has successfully secured billions of dollars in digital assets.
- **Technical Expertise:** Our auditors and researchers hold advanced degrees in cryptography, cybersecurity, and distributed systems, backed by years of hands-on experience.
- **Innovative Research:** MixBytes actively contributes to blockchain security research and open-source tools, sharing insights with the global community.

Our Services

- **Smart Contract Audits:** Comprehensive security assessments of DeFi protocols to detect and mitigate vulnerabilities before deployment.
- **Blockchain Research:** In-depth technical studies, economic and protocol-level modeling for Web3 projects.
- **Custom Security Solutions:** Tailored frameworks and advisory for complex decentralized systems and blockchain ecosystems.

Contact Information

https://mixbytes.io/

https://github.com/mixbytes/audits_public

https://x.com/MixBytes

hello@mixbytes.io